

CS 3960 (3)

Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

UPCOMING

- Due this Friday @ 5pm:
 - Two reading responses
 - Both are in the “Reading 1B” assignment
 - One set of release notes
- No class on Monday the 19th
- Demo Day on Weds the 21st, please be ready

Today: Testing

“Program testing can be a very effective way to show the presence of bugs, but it is hopelessly inadequate for showing their absence”

— Dijkstra, 1972





Why test?

Testing is the reason software works, period.

Tests are also a roundabout way to write software specifications: “Our system is whatever passes our test suite.”

At some software shops, if someone makes a change that breaks your code, but you didn't have a test for the behavior you relied on, this is not their problem: it's your problem.

**If you liked it, you should have
put a test on it.**

— Google's Beyoncé Rule





John's top-level thoughts on testing:

- You have to do it, so you might as well not be dreadfully bad at it.
- Being a good tester is a superpower.
- Testing is a creative activity, not rote work.
- Testing is a first-class part of the development workflow, not an afterthought.

You're unlikely to understand the AI agent's code as well as you would understand your own code.

Testing was already one of the most important things, now it's even more critical.



Tell the coding assistant:

“Do [task], and also add some tests for the new code, and also don’t come back until all tests pass.”

This can save you a lot of time.





But, if the LLM

1. writes the software
2. and also writes the tests

... then how are we supposed to make sure the software does the right thing?

Questions about testing:

- Who's doing the testing?
- What kinds of testing should we do?
- Where do test cases come from?
- What are some characteristics of good test cases?
- How do we decide if a test passed or failed?
- When do the tests get run?
- What kinds of code are difficult to test?
- How can we design software that's easier to test?
- How many tests are enough tests?
- How can we tell if our tests are doing a good job?

Who tests the software?

Of course, for our purposes,
it's you.

But capable software shops
have dedicated testing teams.

Why?





Testing comes in different levels...

- Unit testing
- Integration testing
- Validation testing
- Acceptance testing

Another kind of test is the regression test:

It makes sure that a bug we fixed, stays fixed.

Why might a bug not stay fixed?





A positive test checks whether valid inputs are handled correctly

A negative test checks whether invalid inputs are handled correctly

It's super common for developers to forget to add negative tests!

Error-handling code is notoriously buggy

Where do test cases come from?



- From the software requirements
- From previous bugs (regression tests)
- From equivalently-behaving regions of the implementation
- From boundary cases in the implementation
- From an LLM
- From random generation

In specialized circumstances, we can test software using **all possible inputs** — this is exhaustive testing.

Example: Testing a decoder for ARM, since all instructions are 32 bits on this ISA.



Another example is IEEE single-precision floats: there are only 2^{32} of them. Your laptop can execute 2^{32} instructions in like 1 second.

1. Use Amp to write a C program to compute the percentage (rounded to the nearest whole number) of single-precision floats that are: NaN, +inf, -inf, +0, -0, +subnormal, -subnormal, +normal and -normal.
2. Then do the same for `expf ( f )`, for every float `f`.
3. Then do the canvas quiz in the module for this week. It's fine to work with a partner, but then **both of you should hand in the quiz.**

This is due before you leave class today.